

Load Balancer — Practical System Design Notes

Hinglish • Common-sense • Interview-focused • 12 core topics + questions

0. Big Picture

Load Balancer (LB) ka simple kaam hai incoming traffic ko multiple healthy servers ke beech distribute karna. Isse ek server par saara load nahi aata aur system horizontal scale kar sakta hai.

Without LB:

Users
↓
Node #1

With LB:

Users
↓
Load Balancer
■ ■ ■ Node #1
■ ■ ■ Node #2
■ ■ ■ Node #3

1. Why Do We Need a Load Balancer?

Agar ek hi Node.js server hai aur traffic suddenly increase ho gaya, CPU, memory, connections ya application capacity bottleneck ban sakti hai. Multiple instances ke saath LB traffic distribute karta hai.

1000 users
↓
Load Balancer
■ ■ ■ 333 → Server 1
■ ■ ■ 333 → Server 2
■ ■ ■ 334 → Server 3

Interview question: Why not just use one powerful server?

Because vertical scaling has limits and can create a single point of failure. Multiple instances provide horizontal scaling and better fault isolation.

2. What Does a Load Balancer Actually Do?

LB request receive karta hai, configured routing logic ke basis par backend choose karta hai, request forward karta hai, response client tak return karta hai, aur commonly health checks ke basis par unhealthy instances ko traffic se remove karta hai.

Client
↓
LB
■ ■ choose backend
■ ■ check health/status
■ ■ forward request
↓
Server

Question: Kya LB business logic karta hai?

Normally LB ka primary job routing/traffic management hai. Business logic application servers handle karte hain.

3. L4 vs L7 Load Balancing

	L4	L7
Works at	Transport layer	Application layer
Typical signals	IP + port + TCP/UDP connection	HTTP method, host, path, headers, cookies, etc.

Example	TCP 443 traffic	/api → API servers
Deep HTTP routing	No	Yes

Common-sense: L4 ko bolo 'connection-level traffic distribute karo'; L7 ko bolo 'HTTP request dekh kar intelligent routing karo'. L7 can route based on URL path or hostname, for example /api versus /images.

Question: /api requests Server Group A aur /images requests Server Group B bhejne hain. L4 ya L7?

Answer: **L7**, because it understands HTTP/application-level information such as path.

4. Round Robin

Round Robin simple rotation hai. Requests sequentially servers ko milti hain.

Servers: A, B, C

Request 1 → A
Request 2 → B
Request 3 → C
Request 4 → A
Request 5 → B

Useful when servers ki capacity roughly similar ho aur requests ka workload roughly comparable ho.

Question: Kya Round Robin hamesha equal load guarantee karta hai?

No. Equal request count ≠ equal work. Ek request 10 ms ki ho sakti hai aur doosri 10 seconds ki.

5. Least Connections

LB active connections ka count dekhkar relatively kam busy server ko next connection/request de sakta hai, depending on protocol and LB behavior.

Server A → 20 active connections
Server B → 5 active connections
Server C → 12 active connections

Next connection → likely Server B

Long-lived connections (WebSocket, streaming, long requests) wale systems mein connection-aware routing useful ho sakta hai.

Question: Round Robin ke instead Least Connections kab useful ho sakta hai?

Jab requests/connections ki duration significantly vary karti ho aur current active connection count load ka better signal ho.

6. Health Checks

LB periodically ya according to its configuration backend health check karta hai. Unhealthy instance ko new traffic se temporarily remove kiya ja sakta hai.

LB
■■■ Server 1 → /health → 200 ✓
■■■ Server 2 → /health → 200 ✓
■■■ Server 3 → timeout ✗

Traffic:
Server 1 ✓
Server 2 ✓
Server 3 ✗ (temporarily out)

Health endpoint lightweight hona chahiye. Liveness aur readiness concepts alag ho sakte hain: process alive hona aur traffic serve karne ke liye ready hona same baat nahi.

Question: Agar ek Node server crash ho gaya toh LB kya kar sakta hai?

Health check fail hone par LB us instance ko new traffic se remove kar sakta hai, while healthy instances continue serving.

7. Sticky Sessions

Sticky/session affinity ka matlab hai ek client ko repeatedly same backend instance par route karne ki koshish karna.

User A
↓
LB
↓
Server 1

Later:
User A
↓
LB
↓
Server 1 (sticky)

Problem: agar Server 1 par in-memory session hai, scaling/failure ke time coupling create hoti hai. Modern scalable systems often prefer stateless application servers with shared session state in Redis/database when session state is actually needed.

Question: Sticky sessions ko default solution kyun nahi banana chahiye?

Because traffic can become uneven and server-specific state makes scaling/failover harder. Stateless servers generally scale more cleanly.

8. Session Problem with Multiple Servers

Login:
User → LB → Server 1
↓
session in RAM

Next request:
User → LB → Server 2
↓
session missing ■

Better pattern: keep application instances stateless and store shared session state in a centralized/shared system such as Redis or a database, or use an appropriate token-based design.

Question: Why does JWT often simplify horizontal scaling?

If the access token is self-contained and validation does not require server-local session state, any healthy application instance can validate it. This does not make JWT automatically stateless in every architecture; token revocation/refresh and server-side state can still exist.

9. Reverse Proxy vs Load Balancer

They overlap in capabilities, so terminology can be confusing. A reverse proxy sits in front of backend servers and can provide TLS termination, routing, caching, compression, security controls, etc. A load balancer focuses strongly on distributing traffic across backends. A single product can do both.

Client
↓
Reverse Proxy / Load Balancer
↓
Backend servers

Nginx can act as a reverse proxy and load balancer. Cloud/provider LBs can provide managed health checks, scaling integrations, TLS, and other features.

Question: Is every reverse proxy a load balancer?

No. A reverse proxy can exist without distributing traffic across multiple backends.

10. Load Balancer + Auto Scaling

LB distributes traffic; Auto Scaling changes the number of backend instances based on configured signals/policies. They solve related but different problems.

Normal traffic:

LB → [Server 1, Server 2]

Traffic rises:

Auto Scaling



add Server 3, Server 4

LB

■ → Server 1
■ → Server 2
■ → Server 3
■ → Server 4

When a new instance becomes healthy/registered, the LB can begin sending traffic to it according to the platform's registration and health-check process.

Question: Does a load balancer itself create new servers?

Usually no. Auto Scaling/compute orchestration creates or removes instances; the LB routes traffic to the healthy registered targets.

11. Load Balancer Failure

A single LB can itself become a single point of failure. Managed cloud load balancers are typically designed as highly available distributed services, often spanning multiple Availability Zones.

Users



Managed LB

■■■ AZ-1 → servers
■■■ AZ-2 → servers

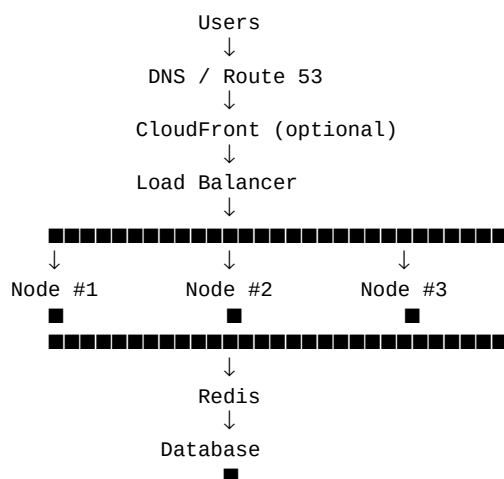
If one zone has trouble,
healthy capacity in another zone can continue
according to the service architecture/configuration.

For self-managed Nginx, you need to design LB redundancy yourself. Don't assume 'I have two Node servers' automatically makes the LB highly available.

Question: Why deploy across multiple Availability Zones?

To reduce dependence on a single zone/failure domain and maintain service availability when one zone has an issue.

12. Real Production Architecture



■■■→ S3 (files/media)

Typical responsibilities: CloudFront handles edge caching/delivery; LB distributes application traffic; Node instances run business logic; Redis handles shared cache/session/rate-limit use cases; database stores durable application data; S3 stores objects such as images/videos/documents.

13. High-Value Interview Questions

Question	What interviewer is testing
Why do we need a load balancer?	Horizontal scaling, availability, traffic distribution
L4 vs L7?	Networking vs HTTP-aware routing
Round Robin vs Least Connections?	Routing strategy and workload shape
How does LB know a server is dead?	Health checks
What happens if a server crashes during traffic?	Failover/routing behavior
Why can sticky sessions be problematic?	Statefulness and uneven distribution
How do you share sessions across instances?	Redis/DB/token-based approaches
Reverse proxy vs LB?	Roles and overlap
How does Auto Scaling interact with LB?	Dynamic capacity + registration
Can the LB itself fail?	HA and failure domains
Where would CloudFront fit?	CDN vs application LB
Why use S3 for uploaded files?	External durable object storage vs local disk
What if traffic jumps 10x?	Autoscaling, capacity, caching, bottlenecks

14. Final Cheat Sheet

Load Balancer

→ Distributes incoming traffic

L4

→ IP/port/connection level

L7

→ HTTP-aware routing

Round Robin

→ Rotate requests across servers

Least Connections

→ Prefer server with fewer active connections

Health Check

→ Remove unhealthy targets from new traffic

Sticky Session

→ Try to keep client on same backend

Stateless App

→ Any healthy server can handle the request

Redis / shared store

→ Shared state when needed

Reverse Proxy

→ Front door to backend; may also load balance

Auto Scaling

→ Add/remove backend capacity

HA Load Balancer

→ Avoid LB as a single failure point

CloudFront

→ CDN/cache; usually before application LB for suitable traffic

S3

→ Object storage for files/media

15. Practice Prompts

After reading this PDF, practice answering these in 30–60 seconds each:

- A. Why can't we simply run one very powerful Node.js server?
- B. Explain L4 vs L7 using one real example.
- C. Why does Round Robin not always mean equal load?
- D. What happens when one backend becomes unhealthy?
- E. Why can sticky sessions make scaling harder?
- F. How would you handle login/session state with three Node.js servers?
- G. What is the difference between a reverse proxy and a load balancer?
- H. How do Auto Scaling and the LB work together?
- I. How would you design the system so the LB is not a single point of failure?
- J. Where would CloudFront and S3 fit into the architecture?

Next practical step: Build this locally with 3 Node.js instances, one Nginx load balancer, a health endpoint, and a small request counter. Then observe Round Robin, failure handling, and sticky-session behavior yourself.